



Rapport de stage

Creation of a 360° Navigation Dataset Using 3D Gaussian Splatting

effectué par *CARMOUZE Guilhem Antoine Fernand*

du 15 avril 2026 au 21 août 2026

Tuteur de stage : Yusuke Yoshiyasu (吉安 祐介), Computer Vision Research Team Leader

Superviseur académique : Florent Lamiraux, Directeur de recherche CNRS, LAAS-CNRS

Établissement : Université de Toulouse. Diplôme préparé : cycle ingénieur UPSSITECH, 4ème année – Spécialité SRI (2025-2026).

Entreprise d'accueil : National Institute of Advanced Industrial Science and Technology (AIST), Tsukuba, Japan

Résumé

Ce stage, réalisé à l'AIST, porte sur la création d'un jeu de données destiné à la navigation visuelle robotique. L'objectif est d'associer, à chaque instant d'un déplacement, ce que le robot voit, sa position, son orientation et la commande qu'il exécute. Ces séquences peuvent ensuite servir à entraîner ou à évaluer un modèle de navigation qui choisit ses actions à partir de ses observations.

Une simple vidéo ne répond pas complètement à ce besoin. Elle ne montre le lieu que depuis la trajectoire et les directions choisies lors de sa capture. Or, pour entraîner ou tester un robot, il est utile de pouvoir rejouer un même trajet, modifier sa vitesse, observer une intersection dans une autre direction ou créer une nouvelle trajectoire tout en conservant un environnement visuellement réaliste. Le 3D Gaussian Splatting (3DGS) offre cette possibilité : il reconstruit une représentation tridimensionnelle à partir d'images, puis permet de rendre rapidement de nouvelles vues depuis des positions et orientations choisies. Il devient alors possible de simuler une trajectoire et d'enregistrer les observations correspondant précisément aux besoins du jeu de données.

Dans un 3DGS, la scène est décrite par de nombreuses primitives gaussiennes anisotropes, dont la forme peut être étirée et orientée dans l'espace. Chaque primitive possède une position, une orientation, une étendue, une opacité et une apparence qui peut varier avec la direction d'observation. Lors du rendu, ces primitives sont projetées dans l'image puis composées selon leur profondeur et leur opacité. Le résultat peut être photoréaliste lorsque les images d'origine couvrent bien le lieu, tout en restant suffisamment rapide pour simuler une caméra en mouvement.

Une scène 3DGS est construite à partir d'images qui se recouvrent. Les positions et orientations des caméras sont d'abord connues ou estimées à partir des correspondances entre ces images. Un nuage de points initialise ensuite les Gaussiennes, dont les paramètres sont optimisés afin que les vues rendues reproduisent les images de référence. Le modèle obtenu peut être exporté dans un fichier .ply contenant les paramètres des Gaussiennes. Ce fichier n'est pas un maillage, c'est-à-dire une surface continue formée de triangles : il est très adapté au rendu de nouvelles vues, mais ne garantit pas à lui seul des murs continus, un sol praticable ou une géométrie assez fiable pour déterminer où le robot peut circuler.

Au début du stage, le laboratoire avait déjà fourni le .ply d'une scène intérieure complète. La première étape consistait donc à y simuler le déplacement d'un robot et à enregistrer des données synchronisées. Une représentation de l'espace libre permettait de calculer une trajectoire, puis un contrôleur la transformait en commandes continues. À chaque position, l'image, la position, l'orientation et les commandes étaient enregistrées ensemble afin de relier l'observation au mouvement réellement simulé.

Il restait à choisir quelle image enregistrer. Une vue perspective vers l'avant est suffisante pour suivre un couloir, mais elle perd des informations utiles lorsque le robot tourne, atteint une intersection ou doit retrouver un objectif situé hors de son champ de vision. Le projet utilise donc un panorama équirectangulaire (ERP), une image rectangulaire qui rassemble l'ensemble des directions autour du robot sur 360°. Depuis ce panorama, une vue perspective peut ensuite être extraite dans la direction du déplacement ou vers un objectif visuel.

Le moteur de rendu produisant des images en perspective, un panorama était construit à partir de six vues prises depuis le même centre optique et orientées selon les faces d'un cube.

Ces vues étaient reprojétées dans l'ERP ; un léger recouvrement et une pondération près de leurs bords réduisaient les raccords. Cette chaîne a produit une séquence synchronisée de panoramas et de données de navigation. Un post-traitement a atténué certains défauts visuels sans modifier la structure de la scène. Le résultat restait néanmoins dépendant du .ply fourni.

La suite du stage visait à supprimer cette dépendance en partant d'une vidéo enregistrée par une seule caméra en mouvement. Deux voies ont alors été étudiées. La reconstruction cherche à retrouver les caméras et la scène à partir des images réellement capturées. Elle est plus fidèle au lieu observé, mais devient moins fiable dans les zones peu ou jamais vues. La génération cherche au contraire à compléter ces régions absentes en produisant un contenu plausible. Elle peut étendre les vues disponibles, mais risque aussi d'inventer une ouverture, un obstacle ou une organisation spatiale différente du bâtiment réel. Ces deux approches répondent donc à des besoins complémentaires, sans garantir à elles seules une scène directement utilisable pour la navigation. Générer directement le panorama à partir de chaque image perspective, sans fondation 3D, a été écarté : la génération y serait peu contrainte (au risque d'inventer, par exemple, plusieurs cuisines dans un même logement), sans cohérence temporelle garantie, et les vidéos ERP d'intérieur nécessaires pour entraîner un tel modèle sont rares.

Enfin, plusieurs approches ont été examinées pour améliorer des rendus incomplets ou dégradés, l'objectif étant de réduire les défauts visuels tout en conservant une représentation suffisamment cohérente pour produire des panoramas de navigation. Les expériences ont montré que la réparation améliore souvent chaque vue prise isolément, mais qu'elle peut dégrader l'accord entre les directions d'un même panorama : des images plus propres ne forment pas nécessairement un environnement unique et stable dans le temps. Ce diagnostic a conduit, en fin de stage, à une chaîne finale contrainte par la géométrie : un unique flux vidéo continu balayant toutes les directions est réparé, puis redistillé dans la scène 3D partagée sous contraintes de profondeur et de recouvrement, avant le rendu panoramique final. Cette chaîne a produit, à partir d'une simple vidéo perspective, une vidéo ERP complète dont la cohérence entre les directions et dans le temps a été constatée qualitativement ; sa validation quantitative reste à mener. Le stage aboutit ainsi à trois résultats principaux : le prototype fonctionnel sur scène fournie, l'identification des conditions de validité (géométrie partagée validée, traçabilité de l'origine des contenus, espace libre vérifié indépendamment) et cette chaîne finale opérationnelle. Une poursuite du travail est proposée autour d'une validation géométrique préalable et d'un modèle de réparation entraîné directement sur des séquences panoramiques.

Mots-clés : 3DGS, navigation, panorama à 360°, reconstruction, génération de monde

Abstract

During my internship, I worked on a 360° visual-navigation dataset in which each image is linked to the position, orientation and motion of a robot. Instead of recording every possible route with a physical camera, I used 3D Gaussian Splatting (3DGS), a photorealistic scene representation that can quickly render new viewpoints. This makes it possible to choose a trajectory in a virtual scene and generate the observations required for navigation.

The first prototype used a complete 3DGS scene supplied by AIST. At each simulated robot position, six perspective views were combined into an equirectangular panorama (ERP), an image that covers all directions around the camera. The panoramas, robot states and motion commands were recorded together. A later image-processing step reduced some visible defects, while depth remained a separate proof of concept.

The second part of the work examined how to remove the need for a prebuilt scene. I studied both reconstruction, which tries to preserve the place shown in an ordinary video, and generation, which can complete directions that were never recorded. I then examined ways to improve incomplete renders. The experiments showed that cleaner individual images do not necessarily form one consistent environment across directions and time. This diagnosis led to a final geometry-constrained pipeline in which one continuous video stream covering all directions is restored and distilled back into the shared 3D scene before panoramic rendering, producing complete ERP video that appeared coherent across directions and time under qualitative inspection. The internship therefore produced a working supplied-scene prototype, a diagnosis of the main failure modes, and this first video-to-ERP pipeline, whose quantitative validation remains to be completed.

Keywords: 3DGS, navigation, 360° panorama, reconstruction, world generation

SOMMAIRE

INTRODUCTION.....	6
1- INTERNSHIP CONTEXT AND RESEARCH OBJECTIVE.....	7
2- TECHNICAL BACKGROUND AND EVALUATION FRAMEWORK.....	8
3- NAVIGATION AND ERP RENDERING FROM A SUPPLIED 3DGS SCENE.....	12
4- CONSTRUCTING THE SCENE FROM CONVENTIONAL VIDEO.....	14
5- NAVIGATION-ORIENTED EVALUATION OF SCENE-CREATION METHODS.....	16
6- PANORAMIC REPAIR AND GEOMETRY-CONSTRAINED RESTORATION.....	20
7- DISCUSSION, VALIDITY CONDITIONS AND PROPOSED CONTINUATION.....	24
CONCLUSION.....	26
BIBLIOGRAPHY.....	27
WEBOGRAPHY.....	28
APPENDICES.....	29

Introduction

I carried out this research internship at AIST in Japan from 15 April to 21 August 2026 as part of the Systèmes Robotiques et Interactifs (SRI) engineering programme at UPSSITECH. My task was to produce visual data for training and evaluating a robot-navigation model.

An image only becomes useful for navigation once it is tied to the robot's position, orientation and movement at that same instant. A forward-facing camera gives one direction at a time, and during a turn or at an intersection what matters is often beside the robot or behind it. I used equirectangular panoramas (ERP) instead: rectangular images that cover the full 360° around one position, from which a perspective view can be cut out afterwards, either along the direction of travel or towards a visual goal.

Recording the data directly with a physical robot would restrict the dataset to the few interiors the laboratory can physically access, and inside each of them to the paths actually driven. 3D Gaussian Splatting (3DGS) offered a way around this. It represents a photographed place in three dimensions and renders photorealistic views quickly from new camera positions [1], so once a scene exists, any trajectory, speed or viewing direction can be simulated.

I started from a complete indoor 3DGS scene supplied by AIST and used this controlled case to build the navigation and panoramic-rendering pipeline. The harder question came next: how to obtain such a scene from an ordinary video. Two routes were available — reconstructing what the camera had seen, or generating what it had not — and I studied both before asking whether incomplete renders could simply be repaired.

One difficulty runs through the whole project: the gap between looking right and being right. A panorama can look complete and still show contradictory walls or openings in different directions, while a faithful reconstruction can be full of holes as soon as the camera leaves the recorded path. For the dataset to be usable, the images, the positions, the commands and the free space must describe the same environment — and that environment must still be the same when the robot comes back to it.

Research question: How can coherent ERP navigation observations be produced from a 3DGS scene, first when a complete scene is supplied and then when it must be reconstructed or generated from limited perspective video?

The report follows the same order as the work: the supplied-scene prototype and the panoramas it produced, then reconstruction and generation from video, the link between this data and robot navigation, the limits of image repair, and finally the geometry-constrained pipeline that came out of that analysis, together with the validation rules I would apply next.

1- Internship context and research objective

1.1 Host organisation

I completed the internship at the National Institute of Advanced Industrial Science and Technology (AIST) in Japan from 15 April to 21 August 2026. AIST is a Japanese public research organisation whose activities cover fundamental research, technological development and transfer to industry [W1]. My supervisor was Yusuke Yoshiyasu, whom AIST lists as the leader of the Computer Vision Research Team within its Artificial Intelligence Research Center [W2]. The team works on computer vision, three-dimensional modelling and robot learning. That mix explains the shape of the project: rendering, reconstruction and robot navigation together, with a virtual environment providing repeatable data instead of a physical robot run for every experiment. No solution was fixed in advance, so I worked by successive prototypes, the result of each one deciding the next experiment.

1.2 Scientific approach and reproducibility

I presented my progress to the team in weekly meetings. The report follows the chronological sequence of the work.

The work progressed through five main phases: navigation and panoramic rendering in the supplied scene, reconstruction from ordinary video, comparison with scene-generation approaches, improvement of incomplete renders, and the design of a final geometry-constrained restoration pipeline. The software and methods used in each phase are introduced only when their role becomes necessary. I ran experiments both locally and on AIST computing resources, then checked the results against saved images, videos, measurements and validation scripts.

Before larger experiments, I checked the transformation that converts coordinates between the camera and the scene, the coordinate axes used by the rendering and computer-vision tools, and the conversion between perspective and spherical images. An inverted transformation or a wrong axis convention still produces a perfectly plausible image, with the camera on the wrong trajectory. A program that runs without an error proves nothing on its own.

1.3 Target dataset and required metadata

In the prototype, each navigation sample corresponded to one camera pose, meaning one position and orientation of the simulated robot. Table 1 lists the fields I aimed to store. I demonstrated synchronised colour images, poses and motion commands. Depth and collision information remained separate outputs because their conventions and geometric validity still required independent checks.

Table 1: Intended content of one navigation sample. Source: internship specification.

Field	Purpose
360° colour panorama (ERP)	Complete visual context around the agent
Panoramic depth	Distance to visible surfaces
Camera pose	Position and orientation in one world frame
Linear and angular commands	Motion that connects successive observations
Occupancy or collision data	Physical validity of the trajectory
Timestamp or sample index	Synchronisation of all data types

Across a complete sequence, I also needed a scene identifier, the trajectory, the camera calibration, the pose and depth conventions, and a provenance label distinguishing directly observed data from reconstructed or generated content, so that generated geometry is not mistaken for a measurement. Colour, depth and commands must also refer to the same virtual instant.

2- Technical background and evaluation framework

2.1 3D Gaussian Splatting and geometric limits

3DGS represents a scene with many three-dimensional Gaussian primitives [1]. Each primitive has a centre, an elongated shape and orientation, an opacity that controls its transparency and an appearance. When a view is rendered, these primitives are projected as ellipses and blended in depth order. This representation supports rapid photorealistic rendering along a trajectory.

Constructing the representation requires overlapping photographs with known or estimated camera calibration and poses. A sparse set of matched three-dimensional points initialises the Gaussians, whose parameters are then adjusted by comparing rendered views with the original photographs, while primitives may be added, divided or removed. The trained representation can be exported as a .ply file that stores the Gaussian parameters.

This file is not a triangle mesh with connected surfaces. It therefore does not guarantee closed floors or walls. Misplaced or semi-transparent Gaussians may reproduce the original images while giving incorrect depth or incorrect surfaces for determining whether the robot would hit an obstacle. Weakly observed regions can also contain holes or isolated floating elements, commonly called floaters, which become visible after a translation or a large rotation. So I did not give the laboratory's complete .ply and a scene reconstructed from a narrow camera path the same level of confidence.

2.2 Why ERP is useful for navigation

I chose ERP because a forward-facing sequence loses useful directions during turns and at intersections. One panorama preserves the full sphere at each pose and can be converted back into a conventional perspective crop for any horizontal or vertical viewing angle. A navigation model trained on ordinary camera images can therefore receive a view aligned with the

robot's heading, and a panorama recorded at a later position can provide a visual subgoal, an intermediate image that the robot should approach.

Each sample must still record the robot heading, the coordinate conventions of the world, robot and camera, the camera calibration and the angular extent, or field of view, of each extracted crop. ERP also joins the left and right edges of the image because they represent adjacent directions, while the top and bottom of the sphere are stretched in the rectangular image. A method trained only on ordinary perspective images may not handle these properties correctly. These constraints later affected the panoramic-repair experiments.

The renderer only produced perspective images, so the panoramas had to be assembled from six of them.

2.3 Six-view projection from perspective images to ERP

Most Gaussian renderers use perspective cameras. I produced a complete spherical observation with six cameras sharing the same centre and looking forward, right, backward, left, upward and downward. Together, their images form a cubemap, a representation of the surrounding view on the six faces of a cube.

For an equirectangular output pixel (u, v) of width W and height H , longitude λ and latitude φ can be written as:

$$\lambda = 2\pi(u/W - 1/2), \quad \varphi = \pi(1/2 - v/H).$$

These angles define one viewing direction, which is matched to the corresponding cubemap face and sampled. This geometric mapping is independent of the way overlapping images are blended.

Quality control must inspect the join between the left and right edges, the distortion near the poles and the orientation conventions of the six faces; Section 3.3 details the overlap, blending and orientation checks.

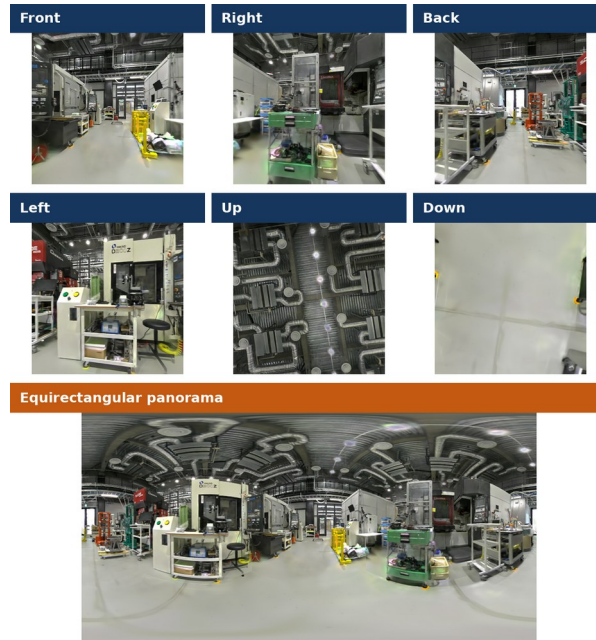


Figure 1: Cubemap-to-equirectangular conversion used in the supplied-scene prototype. All six cameras have the same centre.
Source: author; experimental records.

Figure 1 summarises the chain. It only works if the scene really holds usable information in every direction — which stopped being true as soon as the ready-made .ply was removed.

2.4 Reconstruction, generation and repair

Removing the ready-made .ply changed how the scene had to be obtained. Reconstruction estimates the cameras and the three-dimensional content from images of the real place. It aims to preserve what was recorded, but its result remains limited by the positions and directions present in the input video.

Generation addresses a different part of the problem. It can create plausible content for directions that were never recorded, but this content is not a measurement of the real environment. A generated room may contain an opening, obstacle or spatial arrangement that did not exist. Some approaches combine both ideas by extending an initial reconstruction with generated views.

Repair starts from incomplete or damaged renders and tries to improve their appearance. A repaired image may look better without proving that the underlying surface is correctly placed. For this project, repaired views also had to agree between directions and remain stable during motion. Table 2 summarises how reconstruction, generation and repair are used in the rest of the report. The individual methods are introduced in the sections where they were studied.

Table 2: Complementary roles of reconstruction, generation and repair. The same visual output may combine all three and must preserve provenance. Source: author synthesis.

Operation	Main input	Main objective	Typical risk
Reconstruction	Images or video	Preserve the captured place	Missing content in directions that no input camera observed

Operation	Main input	Main objective	Typical risk
Generation	Text, image or panorama	Create a complete plausible world	Drift from the real layout or duplicated structure
Repair	Incomplete rendered views	Improve weak or empty regions	Inconsistency between directions and time

A more direct alternative would have been to train a generative model to expand each perspective frame into a full panorama, with no three-dimensional foundation at all. I did not take that route. Most of each panorama would then be invention: with no persistent scene behind it, nothing stops the model from putting a different layout behind the camera at every frame — two different kitchens in the same home, for instance — which is exactly what a navigation dataset cannot contain. The completion would also have to stay coherent along the trajectory, and per-frame outpainting carries no temporal context. Finally, training such a model needs a large corpus of real indoor ERP video, and I found nothing at that scale during the internship; that shortage is part of what motivates the dataset in the first place. The 3DGS scene therefore stayed as the geometric foundation, with generation limited to regions without observations.

These three routes can fail in different ways. I kept image fidelity, spherical and temporal consistency, and navigability as separate evaluation dimensions.

2.5 Evaluation dimensions and acceptance criteria

No single image-quality score could serve as the acceptance test, so I kept the criteria apart:

- fidelity to real observations, using image-comparison measures when reference images exist;
- local quality, including holes, blur and floating elements;
- spherical continuity, especially in overlaps and at the equirectangular boundary;
- multi-view and temporal consistency;
- the amount of surrounding space represented by reliable data and its navigability;
- production cost.

For views with a known reference, I used Peak Signal-to-Noise Ratio (PSNR) and the Structural Similarity Index Measure (SSIM), where higher values indicate a closer match; neither proves correctness away from the reference cameras. For generated views, I also used M_{ET}3R, which tests whether pairs of images support a compatible three-dimensional interpretation [13], while the WorldScore benchmark informed my separation between visual quality, consistency and controllability [14]. These measures did not replace direct checks of panorama boundaries, repeated visits or collision geometry.

What counted as acceptable depended on the task: an aligned colour panorama for rendering; stable views away from the source trajectory for reconstruction; and, for navigation, reachable free space, synchronised commands and consistency through time. The supplied 3DGS was the first case judged on these terms.

3- Navigation and ERP rendering from a supplied 3DGS scene

3.1 System architecture and separation of roles

The first case was the most controlled one. AIST had already provided a complete indoor 3DGS .ply with the optimised Gaussian parameters described in Section 2.1, so nothing had to be reconstructed: my job was to turn this visual representation into a time-ordered sequence of robot motion, synchronised metadata and panoramic images.

The 3DGS scene described appearance but did not directly provide a navigation map. To obtain one, I derived an occupancy grid from a horizontal slice at the robot's operating height. Cells intersected by Gaussian primitives were marked as occupied, small isolated components were filtered, and a safety margin was added around obstacles. The start and goal were then checked against the resulting free space. This grid served as a planning heuristic for the supplied-scene prototype; being derived from Gaussian primitives, it was not treated as independent collision ground truth.

The complete processing chain was:

3DGS scene → occupancy map → path planning → velocity control → six rendered views → 360° panorama → optional image correction.

I used the A-star (A*) algorithm to find a path considered free in the derived occupancy grid between the start and goal, then simplified and smoothed it to avoid sharp turns between neighbouring grid cells. DISCOVERSE connected the Gaussian renderer to the simulated robot [5, W3], while MuJoCo provided its state and physical simulation [4, W6]. I kept the planning map, the simulation and the rendering separate on purpose: a photorealistic image says nothing about whether the robot can actually drive through what it shows. Figure 2 shows the resulting map, path and rendered scene.

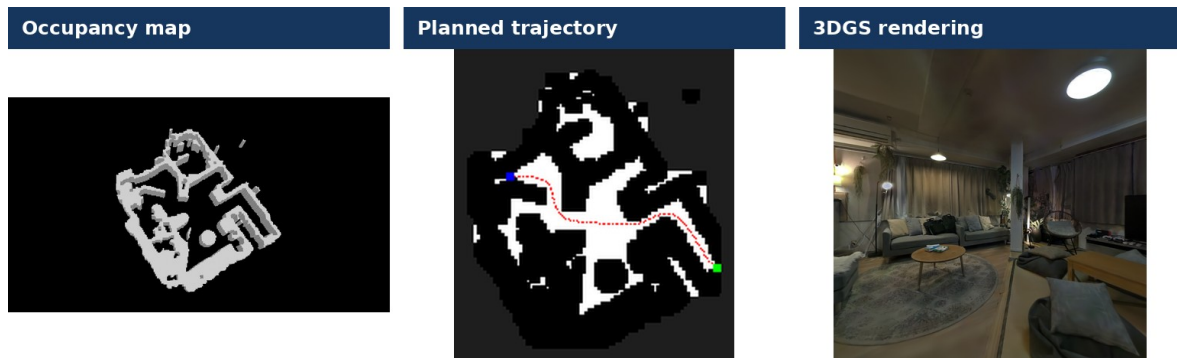


Figure 2: The supplied-scene prototype separates navigation geometry from photorealistic 3DGS rendering. Source: author; experimental records.

3.2 Path planning, control and synchronisation

The planned path described where the robot should move, but it did not provide executable motion commands. For this, I used pure pursuit, a path-following controller that repeatedly selects a target slightly ahead on the path. At each simulation step, the controller converted the direction of this target into linear and angular velocity commands. Forward motion was reduced during large turns, which produced a smooth trajectory and maintained visual overlap between successive observations.

For every selected pose, I stored the timestamp, robot pose, velocity commands and rendered colour observation under the same sample identifier. The coordinate-frame and camera-pose conventions were recorded explicitly so that motion and images could be interpreted in the same reference frame. This synchronisation associated each panorama with the robot state and action that produced it.

3.3 Cubemap rendering and ERP assembly

At each selected pose, I rendered six perspective colour views from the same optical centre and converted them into an ERP using the spherical projection described in Section 2.3. The fixed camera orientations made the conversion repeatable, while the shared centre prevented parallax between the views.

Each view covered slightly more than its nominal cube face, creating a narrow overlap with its neighbours. I combined these overlaps with feathering, which gradually reduced each image's contribution near its border. This reduced visible brightness transitions, but it could not reconcile views that represented different surfaces.

The resulting panorama covered the full sphere. I retained the six source views so that rendering defects could be distinguished from errors introduced during projection or blending.

I also demonstrated panoramic depth separately. However, the archived experiment did not preserve the depth definition, units or convention for missing values. I therefore did not present it as a validated sequence synchronised with the colour panoramas.

The colour-image pipeline produced repeatable panoramas, but black regions, halos and lighting artefacts remained. This motivated the first post-render correction experiment.

3.4 First post-render correction

The archived material calls this first post-processing stage FIXER, but the exact algorithm and version were not preserved. I therefore treat it as a black-box colour-image correction step. It is distinct from the repair method introduced in Section 6.



Figure 3: Aligned panoramas before and after the archived image-correction step. Both panels show the same pose. Source: author; archived experiment videos.

Figure 3 shows that the correction reduced black and smeared regions and weakened the artefact around the ceiling light, although some textures became softer.

Because FIXER operated after ERP assembly, it changed neither the trajectory nor the underlying Gaussian scene. A defect could disappear from the colour image while remaining in the depth map or another viewpoint. The next question was whether the scene itself could be repaired, using geometry taken from somewhere other than the render.

3.5 Validation scope and remaining supplied-scene dependency

The geometric-repair route expected a stable mesh as independent guidance, but no such mesh was available in this experiment. Replacing it with a copy of the damaged Gaussian render provided no new geometric information and made the result worse. I stopped this branch because its required input was missing, without drawing a general conclusion about geometry-guided repair.

This first phase showed that the supplied .ply could support planning, continuous commands, six-direction rendering, deterministic ERP conversion and a colour navigation video. Panoramic depth and the relation between Gaussian opacity and collision geometry remained only partially validated. The limitation was now the starting point itself: everything that worked still rested on a scene someone else had built. The next stage was to obtain a sufficiently complete world without it.

4- Constructing the scene from conventional video

4.1 Reconstruction baseline from conventional video

To remove the dependency on the laboratory's ready-made .ply, I next tried to construct the scene from a conventional video. This required two stages: recovering where the camera had moved, then optimising the Gaussian scene from those estimated views.

For the first stage, I used COLMAP, a structure-from-motion system that detects matching image features and estimates camera positions together with a sparse set of three-dimensional points [2, W4]. It refines both through bundle adjustment, a joint optimisation that reduces the projection error between the images, cameras and points. For the second

stage, I used Splatfacto, the 3DGS implementation provided by the Nerfstudio framework [3, W5]. Splatfacto optimises Gaussian positions, shapes, opacity and appearance. I used 70% of the video frames for this optimisation and reserved 30% for evaluation.

I used the reserved frames to measure quality at viewpoints on or close to the captured route and to compare the two optimisation budgets; I also rotated or translated the camera away from this route and inspected the resulting missing surfaces, floaters and unstable depth, a test kept qualitative since no reference image existed there.

Table 3 shows that the longer optimisation improved the held-out views near the acquisition trajectory. Higher PSNR and SSIM values indicate a closer match to the reserved reference images.

Table 3: Effect of the Splatfacto optimisation budget. Source: baseline experiment records.

Configuration	PSNR $\uparrow$	SSIM $\uparrow$	Approximate time
Splatfacto, 30 000 iterations	26.08 dB	0.91	20 min
Splatfacto, 60 000 iterations	28.69 dB	0.94	40 min

Doubling the optimisation budget improved both reference-image measures, while the approximate computation time also doubled. The views rendered away from the captured route still contained blur, holes and unstable Gaussians, as shown in Figure 4. Pose or optimisation errors may explain part of this degradation, but additional iterations cannot create observations for rays that were never captured.

Reserved frames thus mainly test interpolation near the captured route, whereas navigation may require sideways motion, large rotations or views of unseen surfaces; the dataset must record which of these conditions applies.

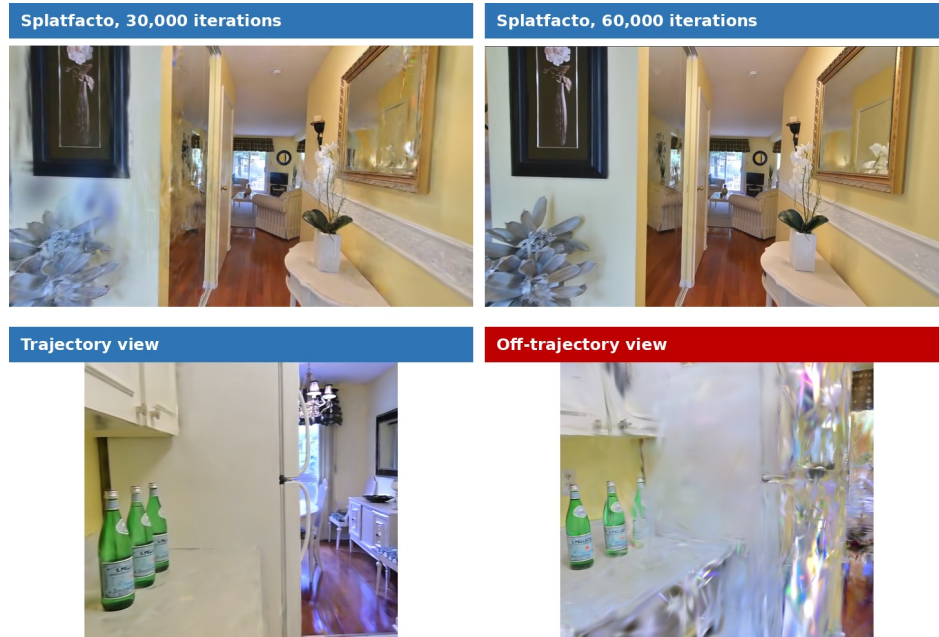


Figure 4: Near-route views from the two Splatfacto optimisation budgets, followed by one trajectory view and one off-trajectory view. The archived slide does not identify which optimisation budget produced the lower examples. Source: author; baseline experiment records.

4.2 Alternative reconstruction methods and remaining limits

The baseline exposed two possible sources of avoidable error: inaccurate camera poses and the difficulty of processing a long video as one coherent reconstruction. I therefore examined methods designed specifically for these two problems.

The archived material also considered MAST3R-SfM followed by 3R-GS for camera-pose refinement, and LongSplat for long videos whose camera positions were initially unknown [6, 7, 17]. More recent feed-forward reconstruction systems such as WorldMirror, which accept heterogeneous priors in a single pass, were identified as further candidates but not tested [19]. I could reproduce only part of the first alternative, while no complete LongSplat run under matching conditions was preserved. Table 4 therefore records the available evidence without using archived scores to rank the methods.

Table 4: Evidence available for the reconstruction routes considered. Source: archived experiment records.

Pipeline	Evidence available	Main conclusion
COLMAP + Splatfacto	Locally reproduced baseline	Reliable mainly near the recorded trajectory
MASt3R-SfM + 3R-GS	Partial local integration	No matched end-to-end comparison
LongSplat	Archived result only	No comparable local run

More importantly, all three approaches remain reconstruction methods. Better camera poses and a better scene representation can reduce avoidable errors, but they cannot recover a wall or a doorway that was never observed in the input video. The question was no longer which method reached the highest PSNR, but which one could give navigation enough space and enough directions to work with — and how the dataset should flag content that was never observed. That is what sent me towards generation methods and navigation-specific criteria.

5- Navigation-oriented evaluation of scene-creation methods

The reconstruction experiments showed that improving image quality near the recorded path was not sufficient. A navigation dataset also needs information when the robot rotates, moves laterally or enters a region that was poorly captured. The next question was therefore not simply which output looked best, but which scene could support the observations and motions required by navigation. Rather than benchmarking world-generation methods for their own sake, I first separated the available scene-creation strategies, then translated the intended use by a navigation model and a physical robot into measurable requirements.

5.1 Reconstruction, generation and hybrid routes

The candidate methods belonged to three families: pure reconstruction, with LongSplat for long videos with unknown initial camera poses [7]; reconstruction supported by generated virtual views, with ExploreGS [18, W11]; and more generative routes, with HY-World 2.0 and PanoWorld [11, 12].

As introduced in Section 2.4, these families give priority either to fidelity to the captured place or to wider coverage. Their inputs were also different, so I did not treat them as interchangeable implementations of one controlled benchmark. Table 5 records why each route was considered.

*Table 5: Main world-creation routes studied. The list is selective and organised by the role each method played in the project.
Source: author synthesis from the cited methods and experiment records.*

Method	Family	Typical input in the study	Relevant contribution
LongSplat	Reconstruction	Ordinary handheld video	Long sequences with unknown initial camera positions
ExploreGS	Reconstruction supported by generated virtual views	Existing 3DGS	Improve explorable weak views
HY-World 2.0	Generation and composition	Text, images or panoramas	Panorama, expansion and 3D composition
PanoWorld	Structured generation	Floor plan and style	Whole-house panoramic layout

These systems were not all reproduced locally. I used their published capabilities to refine the distinction between visual coverage, controllable camera motion and reliable collision support, rather than presenting them as internship results. Only methods for which archived outputs were available appear in the comparison in Section 5.3; PanoWorld was used only to characterise the structured-generation route.

5.2 Intended navigation model and robot

To determine whether the dataset would actually be usable, I examined how its images and metadata would be consumed by a navigation model and transferred to a physical robot. NoMaD is a visual-navigation model that predicts short local motions from a recent sequence of camera images and, when available, an image of the destination [8, W7]. Kachaka is the indoor mobile robot selected as the eventual deployment platform [W8]. Together they turned "navigation-ready data" into a concrete interface whose fields I could actually list.

Unlike the A^* planner and pure-pursuit controller of Section 3, which generated the acquisition trajectories, NoMaD would consume the recorded images and predict motion as short sequences of local waypoints, with or without a destination image; longer routes can be divided into intermediate visual subgoals.

This raised a practical issue for ERP data. The available trained NoMaD weights expect ordinary perspective-camera images, so directly resizing a panorama would change its visual geometry. The planned conversion therefore uses the stored robot heading to extract a perspective crop from the ERP. When an intermediate visual subgoal is needed, it comes from the panorama recorded at the future reference position, not from another direction in the current panorama.

In addition to the common fields in Table 1, NoMaD requires the indices of the recent ERP frames used together by the model, an explicit forward heading and perspective-crop field of view, the goal or subgoal frame indices and headings with the relative pose between the

current and target frames, and a binary mask indicating whether each selected crop contains usable data.

Kachaka set the boundary between the dataset, the model running remotely and the robot itself. The planned software chain was:

camera → image transfer → navigation model → command conversion → Kachaka application programming interface (API).

The image-transfer and basic safety interfaces were prepared, but the final trained model and the conversion from predicted waypoints to robot velocities were not connected. Figure 5 makes this boundary explicit, and I do not claim a complete navigation run on Kachaka.

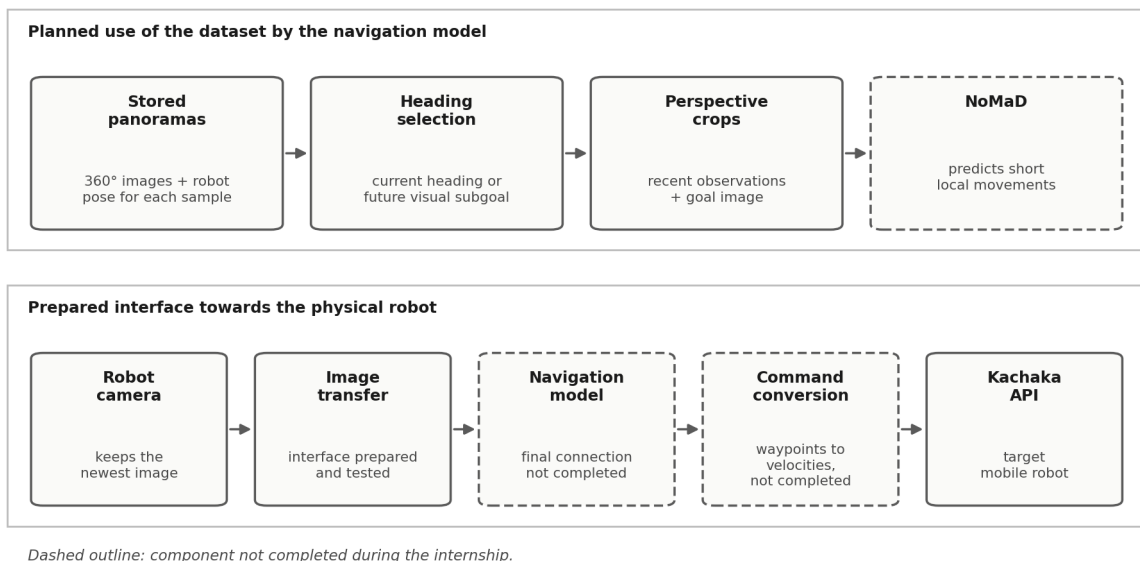


Figure 5: ERP-to-NoMaD data interface and planned Kachaka deployment. The lower row separates the prepared interfaces from the incomplete navigation-model connection and command conversion. Source: author, based on the archived robot-integration code and the NoMaD architecture [8].

Even unfinished, this integration work produced two requirements I could test offline: the observations must keep a stable relation between appearance and motion, and the scene must contain a connected free corridor. The evaluation protocol combined image-quality and consistency measures with near-route interpolation, rotation at a fixed position and lateral extrapolation tests.

5.3 Comparison against navigation requirements

I then compared representative outputs from several reconstruction and generation routes. Matrix-3D is an omnidirectional world-generation method that creates views along a panoramic trajectory before constructing a three-dimensional representation [10, W10]. It was compared with the previously introduced HY-World 2.0 and ExploreGS outputs. The archived study also contained a reference built by organising a Gaussian scene on a spatial grid. Its exact production settings were not preserved, so I label it the Gaussian-grid reference and use it only as a comparison point, not as a reproduced method with the same input conditions.

I retained three measures for the comparison. Spherical coverage is the fraction of viewing directions for which an output is available. The useful radius is reported as an archived relative score. Since the absolute tested distance and failure criterion were not preserved, it supports only a qualitative comparison between these outputs. MET3R p90 is the 90th percentile of the multi-view inconsistency measure introduced in Section 2.5, so a lower value means that even the less favourable view pairs agree better.

I also explored a Gaussian-derived connectivity estimate with NavGSim [15], but the result changed strongly with the conversion thresholds and the assumed robot height. I excluded it from the quantitative comparison and treated navigability as an unresolved validation requirement. Dedicated Gaussian-splatting navigation simulators such as Habitat-GS address this requirement at the simulator level, but they were not evaluated during the internship [16]. Since the methods used different inputs, Table 6 shows trade-offs rather than a controlled ranking.

Table 6: Selected results across three visual and multi-view criteria. Inputs were not matched, so the table supports trade-off analysis rather than a universal ranking.

Method	360° coverage ↑	Normalised useful radius ↑	MET3R p90 ↓
Matrix-3D	0.84	0.67	0.36
HY-World 2.0	0.78	0.59	0.26
ExploreGS	0.97	0.95	0.21
Gaussian-grid reference	0.93	0.95	0.16

Table 6 does not identify one method that is best in every respect. ExploreGS gave the highest coverage and useful radius, while the Gaussian-grid reference gave the lowest multi-view inconsistency. HY-World 2.0 and Matrix-3D offered different compromises, and the reference required a different type of input. Figure 6 places the corresponding outputs side by side. Wide coverage is useful only if neighbouring views agree, while a good MET3R score does not prove that the generated floor plan is correct.

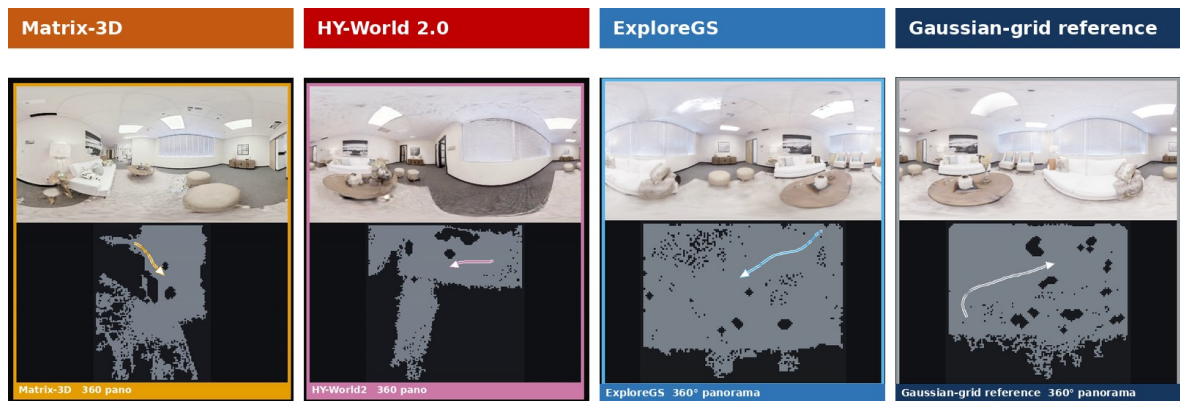


Figure 6: Qualitative outputs from reconstruction and generation routes with different input conditions. Source: consolidated experiment records.

Excluding the provisional connectivity estimate does not remove the need for collision validation from an independent mesh, checked colour-and-depth data or a verified floor plan. It only prevents the Gaussian-derived map used here from serving as collision reference data.

Since none of the evaluated routes resolved both coverage and geometric consistency, I isolated a narrower question: could an image-repair model improve incomplete renders without making agreement between views worse? The next section introduces the model used for this test.

6- Panoramic repair and geometry-constrained restoration

6.1 Rationale and model

The previous experiments had not produced reliable collision geometry, so I set that problem aside and looked at appearance alone. ArtiFixer is a video-repair method that invents missing image content by progressively removing noise, then reuses the frames it has just repaired to continue the sequence [9, W9]. The question I asked of it was narrow: can it clean up weak 3DGS renders without breaking the agreement between views that the dataset needs?

The model regenerates weakly supported regions from noise while keeping strongly supported regions constrained by the original render; opacity, however, does not prove that the geometry is correct, and the method processes one perspective video at a time, so it did not by itself say how several directions could be repaired consistently over a complete sphere.

6.2 Adaptation from perspective video to ERP

The complete scene from the first prototype had views in every direction, which made it useless for studying severely missing content. I ran the repair pipeline on another indoor scene, reconstructed from a moving perspective camera; at one position the original camera had actually seen only about 12% of the surrounding sphere.

The available model had been trained on perspective videos and its reference-view component assumed a perspective camera, so the panorama could not be processed directly.

I instead divided the sphere into several overlapping perspective video streams sharing the same camera centres, each looking in a different direction; ArtiFixer processed each one as a temporal video, the repaired views were projected onto the sphere and combined into an ERP, and pixels supported by the source camera could be reinserted so that generated content did not replace a direct observation.

This kept every model input close to the perspective format it expected, and it created the central problem of the experiment: two streams could look at the same wall and still finish it differently. I therefore measured local improvement separately from agreement around the panorama and stability over time.

6.3 Qualitative and quantitative results

On individual perspective views, ArtiFixer often produced a clear visual improvement. It reduced large black regions and floater-like artefacts, and some damaged renders became plausible room images. Figure 7 compares the raw reconstructed render and the direct ArtiFixer output at the same pose; it demonstrates local appearance differences only.

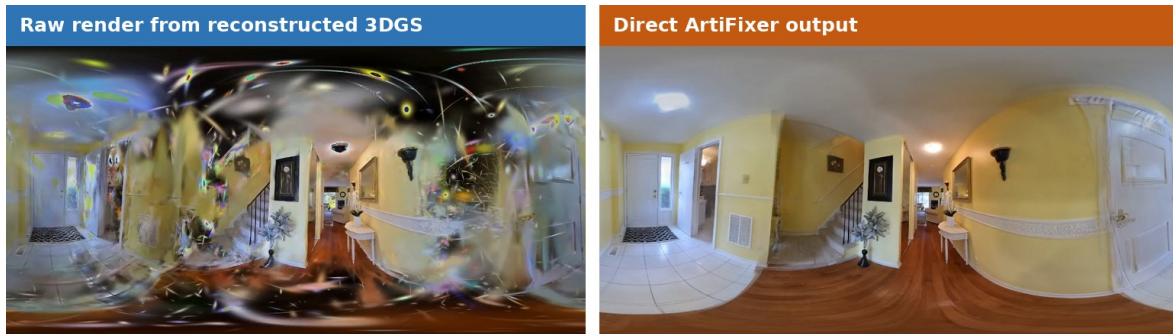


Figure 7: Aligned raw reconstructed 3DGS render and direct ArtiFixer output at the same pose. This single comparison demonstrates local appearance differences, not global or temporal success. Source: author; archived comparison video and provenance record.

At this pose the repaired output is clearly cleaner than the raw render. But one good view says nothing about what the room looks like in another direction, or when the camera comes back to it later.

The repaired output also contains a structural break around the door at the far right of Figure 7: the image is cleaner overall, but the door frame is no longer rigid or continuous.

I used the archived boundary validator to compare neighbouring cubemap faces: a pair of adjacent faces fails when more than 10% of its frames disagree in colour, local structure or feature alignment, and the reported value is the fraction of failing pairs averaged over the sequence. The underlying comparison functions are those fixed in the archived implementation. The mean failure fraction increased from 0.34 before repair to 0.83 after repair, far above the acceptance threshold of 0.10 in both cases: local cleaning worsened compatibility between directions. The values are specific to this scene and are not a general score for ArtiFixer.

Three explanations were possible: the stitching, the mismatch between the trained model and the input, or errors already present in the reconstructed geometry. I looked at each of them.

6.4 Blocking mechanisms

I compared direct cuts, smooth blending and automatic seam placement — these could move or soften a boundary, not reconcile incompatible structures — and tried coordinating the streams through shared initialisation and processing order, without obtaining one shared representation of the scene. Four mechanisms remained:

- Each directional stream reused its own recent frames, so the same wall could be generated differently after a rotation or a revisit.

- Image blending changed the visibility of the boundary but did not correct the incompatible predictions on either side.
- The model expected perspective video, and its perspective-camera component had to be disabled for the panorama experiment.
- Pose errors, weak reconstruction and unobserved directions were all handed to one model asked both to remove artefacts and to invent missing content.

These tests do not allow the failure to be attributed to ArtiFixer alone: the initial reconstruction, the mismatch between perspective-video training and panoramic input, and the independent processing of directional streams all contributed to the result.

6.5 Acceptance decision and scope

Table 7 condenses the final decision based on three requirements: local image cleanliness, compatibility between directions, and stability during motion and revisits.

Table 7: Condensed repair-pipeline decision. Local image improvement was necessary but insufficient for panoramic dataset acceptance. Source: author synthesis from the reported experiment.

Acceptance question	Experimental observation
Are weak individual views cleaner?	Often yes
Do all directions describe compatible rigid structure?	No
Is the same structure stable through time and revisits?	No
Can the result be labelled as one navigation-ready world?	No

I stopped there rather than push this pipeline towards dataset production. The rejection is conditional, not general: geometry has to be validated independently first, and the perspective-versus-panorama mismatch calls for a model built for ERP sequences. The last weeks of the internship used that diagnosis to redesign the pipeline around the shared scene.

6.6 Geometry-constrained panoramic restoration

The redesigned pipeline keeps the reconstructed scene at the centre instead of treating each direction independently. From the input video, COLMAP provides calibrated camera poses and 3DGRUT an initial Gaussian scene shared by every later step [W12]. A single sequence of world-locked overlapping renders — perspective views rendered from camera poses expressed in the fixed coordinate frame of the reconstructed scene, chained into one continuous video that sweeps all directions from the source camera centres — is then restored by ArtiFixer, with the real posed frames as anchors. This removes the first blocking mechanism of Section 6.4: all directions now share one temporal context.

The restored frames are distilled back into the shared scene under depth and overlap-consistency constraints — appearance refined, geometry preserved — and the final panoramas are rendered synchronously from the updated scene, restored by a final pass of ArtiFixer3D+, the 3D-aware extension of the ArtiFixer model [9, W9], and fused into an equirectangular video. Figure 8 summarises the framework.

Framework overview

Pinhole video → geometry-aware restoration → equirectangular 360° video

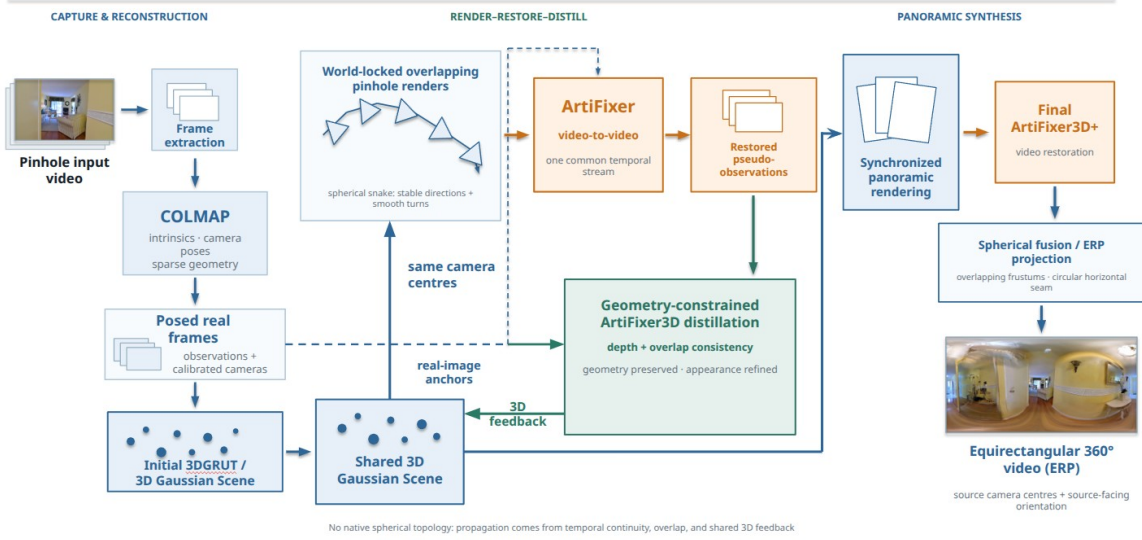


Figure 8: Geometry-constrained restoration framework built around the ArtiFixer models. One continuous render stream is restored, distilled back into the shared Gaussian scene, then re-rendered as a synchronised panoramic video. Source: author.

On the indoor scene of Section 6.2, this pipeline produced complete ERP frames in which walls, openings and objects appeared rigid and mutually consistent through time under qualitative inspection; the first complete run already produced plausible panoramas, and the distillation pass with depth and loop-closure constraints visibly improved the agreement between directions (Figure 9) and improved cross-view depth consistency by 27.3% relative to the first complete run. Apart from this depth-consistency measurement, the assessment remains qualitative: the boundary validator and the navigation criteria of Section 2.5 have not yet been re-applied to these outputs. The pipeline also has no native spherical topology — consistency comes from temporal continuity, overlap and the shared 3D feedback — so a model directly aware of the ERP geometry remains relevant future work (Section 7.3).



Figure 9: Aligned ERP frames from the first complete ArtiFixer3D+ run (left) and after the distillation pass with depth and loop-closure constraints (right). The second output shows improved agreement between directions at the same pose. Source: author; final pipeline outputs.

7- Discussion, validity conditions and proposed continuation

7.1 Diagnostics, discussion and scope of the conclusions

None of the methods taken separately solved the whole problem. This section collects the conditions that stopped each branch, shows how they shaped the pipeline of Section 6.6, and sets out what a complete chain would still have to satisfy.

Three conditions caused an experimental branch to stop. First, independent camera-pose estimates disagreed; the archived record does not preserve the measured disagreement, but it was sufficient to reject the complete reconstruction as reference geometry. Second, increasing visual coverage did not ensure that overlapping views represented the same rigid structure — a panorama could look complete while only a small part was supported by consistent observations. Third, none of the tested repair outputs passed both the checks between neighbouring directions and those performed after motion or a return to the same place. When one of these conditions failed, the branch required better calibration or geometric evidence rather than another image-improvement step.

With the laboratory-supplied 3DGS, I obtained a colour-image navigation sequence combining grid planning, velocity control and ERP rendering; panoramic depth remained separate. Without the supplied scene, none of the individually tested chains produced a persistent world acceptable for navigation; only the final geometry-constrained pipeline, designed from these diagnostics, produced panoramic video that appeared consistent under qualitative inspection, and its validation against the navigation criteria is still pending.

The recurring problem was evidence shared between viewpoints. Reconstruction improved nearby views, generation widened coverage, repair cleaned damaged renders — none of them showed that the same wall was still there from another direction, or after a revisit. ERP makes these contradictions impossible to hide, since every direction ends up in the same image.

These conclusions are limited by one supplied scene, unequal reproducibility, unmatched inputs and one model trained on perspective video. The decision not to use the tested pipelines for dataset production applies only to these conditions. It still shows why geometry and the origin of each region must be checked before image completion, and why fidelity, consistency, coverage and navigability require separate tests.

On this basis, the proposed continuation has two tracks, neither completed during the internship: world validation and panoramic appearance repair.

7.2 Geometry-first validation workflow

I organised the proposed continuation around one validated world:

1. verify the source images and camera poses;
2. construct shared geometry only from consistent observations;
3. check rigid surfaces after revisits and validate traversable free space independently;
4. freeze the accepted geometry and record the origin of every region;
5. render panoramas from this reference and generate content only where no observation is available.

A failure at any validation stage must stop the pipeline rather than being hidden by image completion.

The complete workflow was not implemented during the internship; only the initial source and camera-pose diagnostics corresponding to its first stages were performed (Section 7.1).

7.3 Panorama-aware temporal repair as future work

The second track remains a temporal repair model trained directly on complete ERP sequences. The pipeline of Section 6.6 compensates for the absence of such a model through temporal continuity, overlap and shared 3D feedback, but a model that explicitly represents spherical adjacency could be trained to preserve the same structures across panorama boundaries, frames and revisits. Training it is currently limited by the scarcity of real indoor ERP video noted in Section 2.4 — a gap that the dataset targeted by this internship could itself help to close.

Quality-controlled scenes would provide reference ERP clips along known trajectories, paired with aligned degraded clips from partial reconstructions of the same scenes; reconstructions from real videos remain necessary because artificial degradation may omit failures seen in practice.

Each example should contain the colour panorama, the camera pose, the pixels supported by the reconstruction and the origin of each region. Training must preserve verified observations,

keep completion limited to unsupported regions and maintain agreement through time and revisits, with image quality, panorama boundaries, temporal stability and navigation geometry evaluated separately.

Validated geometry and free-space measurements provide the navigation reference. Panorama-aware repair changes appearance only; never-observed content cannot become reference data for collision or depth.

Conclusion

Three results came out of the internship. With the laboratory scene, the colour-image pipeline ran end to end: path planning, continuous commands, six-view rendering, ERP assembly and post-render correction. Without the initial .ply, none of the individually tested reconstruction, generation or repair pipelines provided sufficient coverage, consistency and navigable support at the same time. The diagnosis of these failures, however, led to the final geometry-constrained pipeline, which produced, from an ordinary perspective video, complete panoramic video that appeared coherent across directions and time under qualitative inspection; its quantitative validation against the navigation criteria remains to be completed.

Studying the navigation model and the robot it would run on turned these results into concrete requirements for images, positions and commands, even though I never completed an end-to-end deployment. The proposed continuation would keep geometry and appearance separate: validated shared geometry would provide the navigation reference, while a panorama-aware temporal model trained directly on ERP sequences would repair appearance. Generated regions must remain identified and cannot become reference data for collisions or depth.

Beyond the technical outcome, this internship built up a set of competences that are worth spelling out.

Scientific and technical competences. The project gave me hands-on experience in 3D vision and novel-view synthesis: structure-from-motion with COLMAP, Gaussian-splatting optimisation with Splatfacto, and above all the analysis of how both fail once the camera leaves the capture trajectory. I implemented spherical-imaging tools, including the cubemap-to-equirectangular projection, overlap blending and boundary validation, and connected them to a robotics stack combining occupancy mapping, A* planning, pure-pursuit control and physical simulation with MuJoCo and DISCOVERSE. I also learned to take recent research code and models, integrate them into an experimental pipeline and adapt them outside their original domain, as in the perspective-to-panorama repair experiment. Debugging coordinate frames and calibration conditioned every single experiment, and checking them became a reflex rather than an occasional fix.

Methodological competences. A program that finishes without an error is not a result. That is what I keep from this internship. The panoramic repair experiment made it concrete: the individual views came back visibly cleaner, while the boundary check went from 0.34 to 0.83 — the images had improved and the scene had fallen apart. From then on I fixed the acceptance

criteria before running an experiment rather than after, kept fidelity, spherical continuity, temporal consistency and navigability as separate questions, and archived the evidence behind each run. It also meant stopping two branches I had already invested time in: the geometry-guided repair, because the reference mesh it needed did not exist, and the repair-centred ERP pipeline, because the measurements no longer supported the claim I wanted to make.

Communication and personal competences. Every week I presented to an international team, in English. Reporting a negative result to that audience is harder than reporting a success: you have to show that the experiment was worth running before you show that it failed. What I learned there was to arrive with the measurement rather than the impression — when I set aside per-frame panorama generation, the argument was not that the method looked wrong, but that nothing would constrain what the model invents behind the camera, and that no corpus of indoor ERP video exists at the scale such a model would need. The subject had no predefined solution, so proposing the next experiment, and arbitrating between local and AIST compute, was my call each week.

Link with the SRI curriculum. These competences directly extend the SRI curriculum, in particular robotics, computer vision and the research training of the TER, and they clarified my interest in the interface between 3D perception and robot navigation. They also form the experience base for the competences reported in my SRI portfolio, on which questions may be asked during the defence.

The next milestone is a short trajectory whose positions, colour panoramas, shared geometry, content origins and navigability pass the same checks. Once this reference sequence is reliable, the remaining model-to-robot interfaces can be completed and the dataset can be scaled.

Bibliography

- [1] B. Kerbl, G. Kopanas, T. Leimkühler and G. Drettakis, “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” ACM Transactions on Graphics, vol. 42, no. 4, article 139, 2023. <https://doi.org/10.1145/3592433>
- [2] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 4104-4113, 2016. <https://doi.org/10.1109/CVPR.2016.445>
- [3] M. Tancik et al., “Nerfstudio: A Modular Framework for Neural Radiance Field Development,” ACM SIGGRAPH 2023 Conference Proceedings, 2023. <https://doi.org/10.1145/3588432.3591516>
- [4] E. Todorov, T. Erez and Y. Tassa, “MuJoCo: A Physics Engine for Model-Based Control,” in 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems, pp. 5026-5033, 2012. <https://doi.org/10.1109/IROS.2012.6386109>
- [5] Y. Jia et al., “DISCOVERSE: Efficient Robot Simulation in Complex High-Fidelity Environments,” arXiv:2507.21981, 2025. <https://doi.org/10.48550/arXiv.2507.21981>

- [6] B. Duisterhof et al., "MASt3R-SfM: A Fully-Integrated Solution for Unconstrained Structure-from-Motion," arXiv:2409.19152, 2024.
<https://doi.org/10.48550/arXiv.2409.19152>
- [7] C.-Y. Lin et al., "LongSplat: Robust Unposed 3D Gaussian Splatting for Casual Long Videos," in Proceedings of the IEEE/CVF International Conference on Computer Vision, 2025. <https://doi.org/10.1109/ICCV51701.2025.02545>
- [8] A. Sridhar, D. Shah, C. Glossop and S. Levine, "NoMaD: Goal Masked Diffusion Policies for Navigation and Exploration," in 2024 IEEE International Conference on Robotics and Automation (ICRA), pp. 63-70, 2024. <https://doi.org/10.1109/ICRA57147.2024.10610665>
- [9] R. de Lutio et al., "ArtiFixer: Enhancing and Extending 3D Reconstruction with Auto-Regressive Diffusion Models," arXiv:2603.00492, 2026.
<https://doi.org/10.48550/arXiv.2603.00492>
- [10] Z. Yang et al., "Matrix-3D: Omnidirectional Explorable 3D World Generation," arXiv:2508.08086, 2025. <https://doi.org/10.48550/arXiv.2508.08086>
- [11] Team HY-World et al., "HY-World 2.0: A Multi-Modal World Model for Reconstructing, Generating, and Simulating 3D Worlds," arXiv:2604.14268, 2026.
<https://doi.org/10.48550/arXiv.2604.14268>
- [12] J. Jia, Z. Li, Y. Hu and Y. Shi, "PanoWorld: A Generative Spatial World Model for Consistent Whole-House Panorama Synthesis," arXiv:2605.17916, 2026.
<https://doi.org/10.48550/arXiv.2605.17916>
- [13] M. Asim et al., "MEt3R: Measuring Multi-View Consistency in Generated Images," in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2025. <https://doi.org/10.48550/arXiv.2501.06336>
- [14] H. Duan et al., "WorldScore: A Unified Evaluation Benchmark for World Generation," in Proceedings of the IEEE/CVF International Conference on Computer Vision, 2025.
<https://doi.org/10.48550/arXiv.2504.00983>
- [15] J. Liu et al., "NavGSim: High-Fidelity Gaussian Splatting Simulator for Large-Scale Navigation," arXiv:2603.15186, 2026. <https://doi.org/10.48550/arXiv.2603.15186>
- [16] Z. Xia et al., "Habitat-GS: A High-Fidelity Navigation Simulator with Dynamic Gaussian Splatting," arXiv:2604.12626, 2026. <https://doi.org/10.48550/arXiv.2604.12626>
- [17] Z. Huang et al., "3R-GS: Best Practice in Optimizing Camera Poses Along with 3DGS," arXiv:2504.04294, 2025. <https://doi.org/10.48550/arXiv.2504.04294>
- [18] M. Kim et al., "ExploreGS: Explorable 3D Scene Reconstruction with Virtual Camera Samplings and Diffusion Priors," in Proceedings of the IEEE/CVF International Conference on Computer Vision, 2025. <https://doi.org/10.48550/arXiv.2508.06014>
- [19] Y. Liu et al., "WorldMirror: Universal 3D World Reconstruction with Any-Prior Prompting," arXiv:2510.10726, 2025. <https://doi.org/10.48550/arXiv.2510.10726>

Webography

- [W1] AIST, "About AIST," official institutional website
https://www.aist.go.jp/aist_e/about_aist/ (accessed 10 August 2026)
- [W2] AIST Artificial Intelligence Research Center, "Computer Vision Research Team," official team page <https://www.airc.aist.go.jp/cvrt/> (accessed 11 August 2026)

- [W3] DISCOVERSE project, official project and code page <https://air-discoverse.github.io/> (accessed 10 August 2026)
- [W4] COLMAP, official documentation <https://colmap.github.io/> (accessed 10 August 2026)
- [W5] Nerfstudio, “Splatfacto,” official documentation <https://docs.nerf.studio/nerfology/methods/splat.html> (accessed 10 August 2026)
- [W6] MuJoCo, official documentation <https://mujoco.readthedocs.io/> (accessed 10 August 2026)
- [W7] NoMaD, official project page <https://general-navigation-models.github.io/nomad/> (accessed 10 August 2026)
- [W8] Kachaka API, official developer guidance from Preferred Robotics <https://kachaka.zendesk.com/hc/en-us/articles/7660222791183-Kachaka-API> (accessed 10 August 2026)
- [W9] ArtiFixer, official NVIDIA research project page <https://research.nvidia.com/labs/sil/projects/artifixer/> (accessed 10 August 2026)
- [W10] Matrix-3D, official project page <https://matrix-3d.github.io/> (accessed 10 August 2026)
- [W11] ExploreGS, official project page <https://exploregs.github.io/> (accessed 10 August 2026)
- [W12] 3DGRUT, official NVIDIA project page <https://research.nvidia.com/labs/toronto-ai/3dgrut/> (accessed 14 August 2026)

Appendices

Declaration of AI assistance

AI was used as a support tool for three tasks only: documentary research, writing code, and helping me correct spelling and typing mistakes in this report.

AI was not used to conduct experiments, produce measurements, invent results or scientific conclusions, or write the report itself. The technical work, the experimental choices, the interpretation of the results and the final responsibility for the submitted report remain my own.